WEEK 9 • REVISED 150-MINUTE GUIDED LAB

# From DRAM behavior to a PYNQ-Z2 SpMV RTL engine

Purpose: NEW material only. Week 7 supplies the CSR algorithm/oracle; Week 8 supplies the CPU/cache/gem5 baseline. Week 9 does not reteach or rerun either.

## 1 / The five-minute handoff

| Carry forward—do not repeat | New question for Week 9 |
|---|---|
| **Week 7: correct integer CSR SpMV; oracle y=[6,6,−1,15]** | Can the RTL produce an identical output, including empty rows and stalls? |
| **Week 8: clustered/scattered traces; gem5 cache/DDR4 timing; independent DDR4/HBM2 replay** | Does the DRAM internal row/bank organization respond differently to new requests? |
| **Week 8: ready/valid accumulator for six precomputed products** | Can RTL fetch x from on-chip RAM, multiply, accumulate indexed rows, and drain results? |

Keep the same SpMV application for project continuity, not as a new algorithm lecture. The Week 8 results are evidence, not fresh Week 9 exercises. Do not compare HBM2 replay time directly to new DDR3/DMA simulations.

## What students learn (new this week)

- DRAM bank, row, activation, precharge and row-buffer behavior in controlled Ramulator experiments.
- How software tile-ordering + a streaming DMA boundary changes which memory requests hardware must handle.
- How a synthesizable SpMV RTL pipeline handles synchronous BRAM, signed MAC, row hazards, stream backpressure and TLAST.
- How to identify the true data-movement boundary for roofline, and what a measured CPU TMAM report can and cannot establish.

## Class schedule: 3 × 50 minutes

| Time | Mandatory task | Evidence |
|---|---|---|
| **0–10 min** | Week 8 handoff and architectural hypothesis | One sentence predicting memory benefit vs host overhead |
| **10–40 min** | DRAM hierarchy experiment (Ramulator) | Actual row-hit/miss/conflict numbers + explanation |
| **40–55 min** | Boundary-aware intensity and optional TMAM demonstration | INTOP/byte with precise counted bytes |
| **55–120 min** | New RTL datapath and Verilator small/full tests | PASS logs, cycle/stall evidence |
| **120–145 min** | PYNQ-Z2 DMA and timing-resource feasibility | Accurate PS→DDR3→DMA→PL diagram |
| **145–150 min** | Submission summary | One improvement hypothesis for Week 10 |

Explicitly out of scope today: CPU cache mapping, reproducing Week 8 gem5 baseline, DDR4-vs-HBM2 replay comparisons, a new CSR algorithm lecture, physical FPGA speedups before synthesis/board validation.

## 2 / Setup: one generator, no simulator installation

Run on server1 in your own authorized course account. The instructor supplies the corrected Week9_SpMV_CoDesign_Lab_Expanded.py generator (Fix 2 source revision) in a read-only classroom directory. Do not use an earlier Week 9 generator; do not use sudo or the instructor login, and do not rebuild the patched Week 8 Ramulator.

Instructor-approved generator SHA-256 (Fix 2): 2614b191ba74744f400ba247a45e2545d37d2e672f5c3aed0f7263e1b9cd3e26. This is the SHA-256 of the Python source, not of the generated workload. If it differs, stop and contact the instructor.

```
source /opt/rh/gcc-toolset-13/enable
export ROOT="$HOME/data/data-intensive/week9"
export COURSE=/home/asinha/data/data-intensive/week9/classroom
export PY=/home/asinha/tools/week8/venv/bin/python
mkdir -p "$ROOT/results"
```

```
cp "$COURSE/Week9_SpMV_CoDesign_Lab_Expanded.py" "$ROOT/"
export W9="$ROOT/Week9_SpMV_CoDesign_Lab_Expanded.py"
"$PY" "$W9" --selftest
sha256sum "$W9" # compare to instructor SHA-256

"$PY" "$W9" --selftest
```

Expected: W9_PYTHON_SELFTEST=PASS tiny=[6,6,-1,15], 8 tiling cases verified. This checks the Python generator and numerical model, not RTL or DRAM operation. The four-row case is an automatic oracle check, not a new CSR teaching exercise.

## Generate ONE medium workload; do not run last week's large comparison again

```
"$PY" "$W9" --make --tiny --out "$ROOT/results/tiny"
"$PY" "$W9" --make --rows 1024 --nnz-per-row 8 \
  --cols 4096 --pattern scattered --tile 2048 \
  --out "$ROOT/results/medium"
```

The medium case (1,024 rows, 8,192 nonzeros) is the compulsory RTL dataset. The full 32,768-row comparison is a Week 10 scaling extension only. This is a newly generated follow-on workload, not a bit-identical rerun of the Week 8 matrices.

## 3 / New experiment: DRAM internal hierarchy (30 min)

Read before executing: each controlled trace makes 4,096 distinct 16-byte reads (65,536 B total). Only access address stride changes: 16 B sequential versus 8,192 B strided. These are isolated synthetic DRAM microbenchmarks, NOT cache measurements, SpMV output traces, real PYNQ DMA captures, or two identical address footprints.

```
set -o pipefail
"$PY" "$W9" --dram-hierarchy --out "$ROOT/results/medium" \
  2>&1 | tee "$ROOT/results/dram_hierarchy.log"
cat "$ROOT/results/medium/w9_dram_sequential_16B.ramulator_metrics.json"
cat "$ROOT/results/medium/w9_dram_strided_16B.ramulator_metrics.json"
```

| Metric | Sequential 16 B | Strided 8,192 B |
|---|---|---|
| **Logical reads / bytes** | 4,096 / 65,536 | 4,096 / 65,536 |
| **Row hits / misses / conflicts** | _____ | _____ |
| **Controller cycles** | _____ | _____ |
| **Mean read latency (if exposed)** | _____ | _____ |
| **Pending callbacks (if exposed)** | _____ | _____ |

Explain ACT (open a row), RD (read), PRE (close an open row), bank-level parallelism, and why address mapping (RoBaRaCoCh), row policy (Open), controller timing and request scheduling determine results. Do not assert that a specific stride must cause a row conflict without interpreting the actual mapper output.

Validity gate: log DRAM model, mapper/row policy, 4,096 requests per trace, drain counters, controller cycles, and genuine row-hit/miss/conflict statistics. Students reproduce their own results; the evidence table remains blank.

## 4 / Co-design: what changed at the memory boundary?

This week's architecture is a teaching-scale, memory-aware streaming engine inspired by the vector-reuse principle in GraphLily (Hu et al., ICCAD 2021). It is NOT a reproduction of GraphLily's HBM engine and does NOT imply state-of-the-art performance on a PYNQ-Z2.

| PS software (outside RTL) | DDR3 and AXI DMA boundary | PL RTL (inside FPGA) |
|---|---|---|
| **Read/validate CSR, group nonzeros by x tile** | MM2S: 64-bit serialized commands | LOAD_X → x_tile BRAM, 2,048×16b |
| **Assemble command stream in DDR3** | Stable TVALID/TREADY and TLAST | MAC: signed 16×16 multiply; indexed y BRAM |
| **Receive y and check full original oracle** | S2MM: 32-bit result stream | CLEAR_Y; DRAIN_Y including empty rows |

Important improvement over Week 8: the host does not compute each product. PS prepares tile-ordered commands, but the FPGA retrieves each x element from its local BRAM and performs multiplication and accumulation. Host bucketing and DMA are real costs, not free accelerations.

## One concrete hardware trace—no re-explaining CSR

Commands are defined by this lab (not a standard AXI packet format): 2-bit opcode, 15-bit row, 11-bit local x index, reserved bits, sign-extended 32-bit immediate. CLEAR_Y zeros y; LOAD_X initializes x BRAM; MAC consumes the stored x entry and matrix value; DRAIN_Y streams all rows.

| RTL operation | Cycle-accurate behavior students must check |
|---|---|
| **LOAD_X** | Write the input x value into local BRAM at the specified local address; accept only on a valid/ready handshake. |
| **MAC_READ** | Capture xmem[local_x] and ymem[row] into x_q and y_q, and register row_q and a_q. The accumulation occurs in MAC_WRITE on the following clock. |
| **MAC_WRITE** | Update y[row] with signed product + old sum; stall input while dependent update completes |
| **DRAIN_Y** | Read y; hold output valid/data/last stable until accepted |
| **CLEAR_Y** | Clear N_ROWS output entries before accepting MAC commands |

Numerical contract: signed 16-bit matrix values/x and exact signed 32-bit intermediate/result. Reject values that violate this narrowed teaching contract; do not silently reinterpret arbitrary 32-bit Week 7 inputs.

## 5 / Mathematical intensity (15 min)

```
"$PY" "$W9" --roofline --out "$ROOT/results/medium"
cat "$ROOT/results/medium/w9_roofline_intensity_proxy.json"
```

Compute INTOP = 2 × NNZ (one integer multiply and one integer add per contribution); approximate input/output command movement = 8 × input-command-count + 4 × N_ROWS. Report INTOP/byte, NOT FLOP/byte. This is a DMA-byte proxy: PS preprocessing, source CSR reads, DDR3 arbitration, cache maintenance and AXI traffic overhead are outside its boundary.

A true platform roofline requires measured sustained bandwidth and correctly matched measured kernel throughput. Do NOT multiply this proxy by a guessed PYNQ bandwidth to claim achieved performance.

## 6 / Core Week 9 task: validate the actual RTL datapath (65 min)

The Week 8 RTL accepted precomputed products. This module must now implement the x-buffer read, signed multiply, y accumulation, zeroing, result draining and AXI-stream stalls. That is the principal graded advance.

## A. Four-row smoke test: 10 minutes

```
"$PY" "$W9" --out "$ROOT/results/tiny" --verilator
```

Required marker: W9_VERILATOR_SPMV=PASS rows=4 mac=6 y=[6,6,-1,15] cycles=39 (instructor server1 preflight, 9 Nov 2026). Do not spend class time manually recomputing the four rows. Changed stimulus may change cycle counts.

## B. File-driven 1,024-row test: 30 minutes

```
"$PY" "$W9" --out "$ROOT/results/medium" --verilator-full
```

Required marker: W9_FILE_RTL=PASS rows=1024 macs=8192 xloads=4096 cycles=23896 (instructor server1 preflight, 9 Nov 2026). The testbench loads cmds.memh/expected.memh, compares EVERY output row, and checks output stability during backpressure. Students explain two pipeline transitions, not generic ready/valid theory. Do not call these physical FPGA cycles.

## C. Select ONE RTL-focused perturbation: 25 minutes

| Choose one | What must be demonstrated |
|---|---|
| **Backpressure: vary output ready** | TDATA/TLAST stable throughout stall; no missing/duplicated results |
| **Command gap injection** | No acceptance without valid && ready; output remains identical |
| **BRAM read latency reasoning** | Identify MAC_READ, MAC_WRITE and the dependency that prevents 1 MAC/cycle here |
| **Row scheduling hazard** | Explain required forwarding/scoreboarding before allowing multiple lanes |

The instructor AND student full tests each observed 23,896 RTL simulation cycles; the generator reported an optimistic lower-bound budget of 23,554 cycles (342 fewer, about 1.45%). At an assumed 100 MHz, 23,896 cycles correspond to 0.23896 ms of modeled core time. The reason for the 342-cycle gap must be supported by RTL signal/counter evidence rather than guessed. These cycles omit preprocessing, physical DMA and FPGA memory-system delay.

### Required evidence: one Verilator PASS log + one short waveform or a 6–10-row cycle table. Answer: where do data wait, where are they stored, and what memory access is avoided?

### Avoid Week 8 repetition

No cache-size sweeps, no four-way DDR4/HBM2 replay, no introductory CSR loop transcription, no fresh X86TimingSimpleCPU baseline. Students only refer back to their previously submitted Week 8 evidence.

## 7 / PYNQ-Z2 deployment path and DMA feasibility (25 min)

The PYNQ-Z2 contains a Zynq-7000 XC7Z020 (PS and PL on one chip) with DDR3 attached to PS, not HBM. A PCIe link is not the accelerator datapath of this project. This week students specify the integration; actual performance claims require Vivado and a successful board run.

| Subsystem | Implementation checkpoint |
|---|---|
| **Zynq PS7** | Use board preset, 100 MHz FCLK0 target, GP0 control and HP0 data access |
| **AXI DMA** | Simple mode; MM2S 64-bit stream commands, S2MM 32-bit results; correct transfer length capacity |
| **Stream RTL** | Connected s_axis/m_axis; proper stable TVALID/TDATA/TLAST, TREADY handling |
| **On-chip memory** | x_tile 4 KiB; y accumulators 128 KiB for 32,768 rows; synthesis must verify BRAM mapping |
| **Vivado reports** | Post-synthesis utilization; post-route WNS/TNS and WHS/THS; bitstream/HWH only after DRC/timing checks |
| **PYNQ runtime** | Allocate DMA buffers, arm RX first, submit TX commands, wait, compare all y against golden file |

There is no PYNQ-Z2 bitstream in this laboratory. The standalone synthetic tiled-DMA DDR3 replay was independently reproduced on instructor and student Linux accounts: 6,401/6,401 requests completed, zero outstanding; 6,145 reads and 256 writes; 26,552 controller cycles (0.049785 ms at 1,875 ps/cycle); 6,343 row hits, 50 misses, 8 conflicts. To reproduce it, run: "$PY" "$W9" --ramulator --out "$ROOT/results/medium", then inspect tiled_dma_16B.ramulator_metrics.json. These numbers are normalized Ramulator controller-replay evidence, NOT measured AXI DMA, FPGA throughput or a board timing result. Actual deployment still requires a synthesized/implemented overlay, bitstream, .hwh, AXI DMA software and board-level correctness measurement.

## 8 / Submit only NEW Week 9 evidence

| Evidence (one compact report + logs) | Rubric /100 |
|---|---|
| **DRAM bank/row metrics; mapper and limitation of stride microbenchmark** | 20 |
| **Correct RTL results, signedness, timing diagram, stable backpressure and counts** | 45 |
| **DMA/PYNQ PS–PL block diagram and BRAM/timing feasibility** | 20 |
| **Boundary-aware INTOP/byte calculation; what is excluded** | 10 |
| **Source/config versions, logs, one Week 10 design hypothesis** | 5 |

This checkpoint belongs inside the existing 30%-of-course guided project; the course weights remain 30% guided / 50% independent / 20% in-class. Required artifacts: DRAM hierarchy metrics JSON for both traces (including row classifications and request accounting), w9_roofline_intensity_proxy.json, tiny and medium RTL PASS logs, an annotated cycle table, and one labeled DMA block diagram. Include source/config versions and distinguish controller replay from physical FPGA measurements.

## Interpretation safeguards

- gem5 Week 8 ROI time is CPU+cache+DDR4. The Week 9 DDR3 request traces are synthetic, normalized standalone Ramulator replays—not a closed-loop PYNQ performance model.
- The 16B memory transactions are a classroom abstraction; do not describe them as captured AXI bursts or physical DDR3 commands.
- No unmeasured DDR3 bandwidth, FPGA Fmax, LUT/BRAM/DSP usage, CPU TMAM percentages or board speedup may appear as measured results.

# 9 / Optional research extensions (not graded this week)

These are useful but intentionally excluded from the mandatory 150-minute guided class so that Week 9 remains an RTL-focused progression.

### Actual Intel TMAM on server1 (optional live demonstration)

Repeat a sufficiently long native SpMV hotspot; use perf stat --topdown only when the Xeon PMU, kernel, privilege and perf version support it. An unprivileged perf or unsupported metric is not a result. CPU Back-End Bound is not automatically DRAM Bound. No four-slot TMAM percentages from gem5 or guessed cache-miss counters.

```
export DIR="$ROOT/results/medium"
g++ -O2 -std=c++17 -DW9_NATIVE "$DIR/week9_spmv_host.cpp" -o "$DIR/spmv_native"
CPU=$(python3 -c "import os; print(min(os.sched_getaffinity(0)))")
perf list --metrics | grep -Ei "topdown|tma_(backend|memory|frontend)" | head
W9_NATIVE_REPEATS=100 taskset -c "$CPU" perf stat --topdown -- \
  "$DIR/spmv_native" baseline 32768 16 262144 scattered 2048
```

Memory-traffic caveat: the native executable generates its own input; whole-process perf includes input preparation and OS effects. Repeats are a useful teaching compromise, not perfect isolated counters. Repeat measurements; label counter support and scope.

### New gem5 PS-bucketing cost (optional follow-on, NOT Week 8 rerun)

The generator also emits week9_spmv_host.cpp and week9_gem5_roi.py. A separate baseline-versus-prepare ROI can estimate CPU-side tile preprocessing. This is a distinct overhead measurement, not the compulsory CPU SpMV baseline. If performed, report the two measurement boundaries and do not add gem5 and independent Ramulator times as if coupled.

### Future Weeks 10–12

- Week 10: sweep x tile size, test reuse versus LOAD_X command count, and measure preprocessing-amortization across repeated SpMV iterations.
- Week 11: explore 2-way MAC issue with correct row hazards/ports and investigate BRAM and AXI stalls.
- Week 12: design-space study with actual synthesis / place / route reports, and physical PYNQ measurements if the board overlay passes.

Other optional, separate studies: NUMA/affinity (native dual-socket), guarded software prefetching, vector move-vs-copy, DMA-buffer coherence/zero-copy, and PCIe accelerator comparison. These are not substitutes for the Week 9 RTL checkpoint.

### References and scientific boundaries

Week 8: Week8_CSR_SpMV_Four_Simulator_Student_Tutorial.docx (carry-forward evidence only); Course syllabus IICD534 Fall 2026; Hu et al., GraphLily, ICCAD 2021; Ramulator2 repository: github.com/CMU-SAFARI/ramulator2; Zynq-7000 SoC TRM (AMD UG585); PYNQ DMA tutorial at discuss.pynq.io/t/tutorial-pynq-dma-part-1-hardware-design/3133. GraphLily inspires a locality principle; PYNQ-Z2 implementation and performance must be independently verified.

FINAL INSTRUCTOR AND STUDENT-ACCOUNT AUDIT (server1, 9 Nov 2026): generator SHA-256 2614b191ba74744f400ba247a45e2545d37d2e672f5c3aed0f7263e1b9cd3e26; Python numerical self-test PASS (8 tiling cases); tiny Verilator PASS (6 MACs, 39 cycles); medium Verilator PASS (1,024 rows, 8,192 MACs, 4,096 x loads, 23,896 cycles); sequential DDR3 replay PASS (4,096/4,096 complete, 17,011 cycles); strided DDR3 replay PASS (4,096/4,096 complete, 57,209 cycles); tiled-DMA replay PASS (6,401/6,401 complete, 26,552 cycles). All three Ramulator metrics JSONs were independently reproduced byte-for-byte at the tested numerical-field level on a separate student account (W9_STUDENT_REPRODUCIBILITY=PASS). All outstanding requests = 0. Simulation

exercise READY for classroom use. Physical PYNQ-Z2 deployment, Vivado timing/utilization and gem5 optional preprocessing experiment have NOT been verified and are not compulsory.